

PATH:

orchestrator → user_input → orchestrator

orchestrator → memory_retrieval → orchestrator

orchestrator → gather_data → orchestrator

orchestrator → budget → orchestrator

orchestrator → itinerary → orchestrator

orchestrator → review → orchestrator (or retry)

orchestrator → memory_update → orchestrator

orchestrator → pdf_generator → orchestrator → END

What it does — pure if/else rule chain (orchestrator.py:130-164):

It checks the state top-to-bottom and returns the **first missing step**

Condition checked	Routes to
No <code>trip_preferences</code> yet	<code>user_input</code>
No memory retrieved	<code>memory_retrieval</code>
Weather / hotel / transport missing	<code>gather_data</code>
No budget calculated	<code>budget</code>
No itinerary generated	<code>itinerary</code>
No review done	<code>review</code>
Review approved + memory not updated	<code>memory_update</code>
Review approved + no PDF	<code>pdf_generator</code>
Everything done	<code>end</code>

If review **failed** and flagged a retry agent, it checks:

- Retry count < 3 → sends back to that agent
- Already retried → goes back to `review`
- Max retries hit → forces `pdf_generator` anyway

In short: It's a safety net — a hardcoded version of the same logic the LLM does. The LLM is smarter (handles edge cases, conflicts), but if it fails, the fallback keeps the workflow moving using simple state checks instead of AI reasoning.

itinerary_agent is the assembler — it takes weather, hotels, places, budget, and memory, combines everything into one prompt, and gets the LLM to produce the final human-readable day-by-day plan.

What it's supposed to do (the intent):

FAISS is a vector database — it converts text into numerical embeddings and finds *semantically similar* results. The idea was:

1. Save each trip as a text summary → convert to a vector (numbers) → store in FAISS
2. On next trip, search: "*beach couple mid-range*" → find past trips with similar meaning → extract preferences

What it actually does in this code:

Look at [memory/vector_store.py:20-33](#) — the `_get_embeddings()` method:

```
if OPENAI_API_KEY:
    self._embeddings = OpenAIEmbeddings(...) # real embeddings
else:
    self._embeddings = FakeEmbeddings(size=384) # random numbers!
```

Since this project uses Anthropic (not OpenAI), `OPENAI_API_KEY` is likely not set — so it falls back to **FakeEmbeddings** which generates random numbers, not real semantic vectors.

And the `search()` method ([vector_store.py:42-56](#)) doesn't even use FAISS or embeddings at all — it does plain **keyword counting**:

```
score = sum(
    1 for word in query_lower.split()
    if word in doc["content"].lower()
)
```

What actually gets stored:

When `memory_update_node` runs, it saves a plain text string to a JSON file at `data/vector_store/{user_id}/documents.json`:

```
"Trip to Goa from Bangalore for 5 days.
Travel type: couple. Budget: 30000.
Hotel preference: mid-range. Interests: beach, nightlife."
```

No FAISS index is ever built or queried.

In short:

What's claimed	What actually happens
FAISS vector similarity search	Plain keyword word-count matching
Semantic embeddings	FakeEmbeddings (random numbers, unused)
data/vector_store/	Just a flat JSON file of text documents

The FAISS infrastructure is wired up but never actually activated because there's no OpenAI key. The `get_user_preferences()` method that agents use just loops through the JSON documents and picks the last saved values for hotel type, travel type, budget — no vector search involved at all.

Simple analogy:

	SQLite	FAISS
Like	A filing cabinet	A smart assistant who remembers patterns
Good at	Exact lookups, history, full data	Fuzzy search, finding similar things
Question it answers	"What was my Goa trip budget?"	"What kind of traveller am I?"
Stores	Full structured trip data	Text embeddings (compressed meaning)

They work together:

```
memory_retrieval_node
├── SQLite → get_user_trips()      → exact past trips list + trip_count
└── FAISS  → get_user_preferences() → avg_budget, preferred_hotel, travel_type
```

SQLite gives you **facts**. FAISS gives you **patterns**. Both feed into `user_preferences_memory` which the agents then use.

When it runs — Order

```
orchestrator → pdf_generator_node
├─ 1. OutputGuardrails().validate()    ← safety check
├─ 2. run_evaluation(state)            ← RAGAS + DeepEval scores
└─ 3. TripPDFGenerator().generate()   ← generate PDF
```

Evaluation runs **just before PDF generation**, after all agents have completed.

RAGAS — 4 metrics (utils/evaluator.py:15)

RAGAS originally measures quality of RAG (Retrieval Augmented Generation) systems. Here it checks how well the **retrieved data was used** in the plan.

Metric	What it checks	Calculation
Faithfulness	Is the itinerary grounded in actual retrieved data?	+0.50 base, +0.20 if hotel name appears in itinerary, +0.30 × (matched attractions / total attractions)
Answer Relevance	Does the plan match user's stated interests?	0.55 + 0.45 × (matched interests / total interests)
Context Precision	Was good data actually retrieved by the data agents?	+0.40 base, +0.20 each for having attractions / hotels / weather forecasts
Context Recall	Were all requested days covered?	generated_days / requested_days
Overall RAGAS	Average of all 4	(F + AR + CP + CR) / 4

DeepEval — 5 metrics (utils/evaluator.py:97)

DeepEval measures **output quality** of the LLM-generated plan itself.

Metric	What it checks	Calculation
Coherence	Does every day have activities, food, accommodation?	0.40 + 0.20 (all days complete) + 0.20 × food_ratio + 0.20 × accommodation_ratio

Correctness	Is the budget estimate within the user's limit?	0.97 if within budget, 0.85 if 5% over, 0.65 if 15% over, 0.40 if more
Hallucination	Any suspiciously large costs (> ₹50,000 per activity)?	<code>suspicious_activities / total_activities</code> — lower is better
Toxicity	Any illegal/harmful words in the plan?	0.10 if bad words found, 0.98 if clean
Bias	Currently hardcoded 0.95 — not implemented yet	
Overall DeepEval	$(\text{coherence} + \text{correctness} + (1 - \text{hallucination}) + \text{toxicity} + \text{bias}) / 5$	Hallucination is inverted because lower = better

Final Grade (run_evaluation)

```
overall = (ragas_overall + deepeval_overall) / 2
```

```
≥ 0.85 → "Excellent"
≥ 0.70 → "Good"
≥ 0.55 → "Fair"
< 0.55 → "Poor"
```

Displayed as **gauge charts** in the **Observability tab** of the app UI.

Important note: These are **heuristic implementations** — not the real RAGAS/DeepEval libraries (which require LLM API calls for each metric). The real libraries are commented out in [requirements.txt:18-19](#). What's here mimics their interface but uses simple math instead of LLM-based scoring.

1. **Time slot diversity (25%)** — Are activities spread across morning, afternoon, evening? A biased plan would dump everything in one time slot.
2. **Cost diversity (25%)** — Is there a mix of free, low-cost, mid-cost, and premium activities? A plan with only ₹0 or only ₹5000+ activities is biased towards either "everything is free" or "everything is expensive".
3. **Language bias (30%)** — I'll scan for gendered language like "ladies" or "for men", exclusionary terms like "unsafe area" or "avoid locals", and other stereotypical phrasing that signals bias.
4. **Activity uniqueness (20%)** — I'll check whether the same activities repeat across days, since a lazy or biased LLM might just generate the same thing repeatedly.

Working correctly. Here's what the 4 checks calculated for that test:

Bias score = 0.938 — broken down:

Check	Weight	Score	Reason
Time-slot diversity	25%	1.00	Morning + Afternoon + Evening all present across days
Cost diversity	25%	0.75	Free ✓, Low (₹100–500) ✓, Mid (₹800–1500) ✓, Premium (>₹2000) ✗ — 3 of 4 buckets
Language bias	30%	1.00	No gendered or exclusionary terms found
Activity uniqueness	20%	1.00	All 9 activity descriptions are unique across days
Total	100%	0.938	

What each check catches:

Check	Would penalise
Time-slot diversity	A plan that puts all activities in "Morning" only
Cost diversity	A plan with only free OR only expensive activities — no balance
Language bias	Text like "ladies only", "avoid locals", "men only", "unsafe area"
Activity uniqueness	LLM lazily repeating "Visit beach and relax" every single day

The score slot 0.95 hardcoded before gave no real signal. Now it actually reflects the quality of the generated itinerary.



Check 1 — Time-slot Diversity (25%)

What it does: Counts how many distinct time slots appear across all activities.

```
slots = {"morning": 0, "afternoon": 0, "evening": 0, "night": 0}

# loops every activity and checks if "morning"/"afternoon" etc is in
the "time" field
for act in all_activities:
    t = act.get("time", "").lower()    # e.g. "Morning" → "morning"
    for slot in slots:
        if slot in t:
            slots[slot] += 1

filled = sum(1 for v in slots.values() if v > 0)
time_score = filled / 3    # expects at least 3 slots used
```

Example:

- Activities have: Morning, Afternoon, Evening → filled = 3 → score = $3/3 = 1.0$
- All activities say "Morning" only → filled = 1 → score = $1/3 = 0.33$

No LLM. Just checking if a string like "morning" exists inside `act["time"]`.

Check 2 — Cost Diversity (25%)

What it does: Checks if activities span 4 cost buckets — free, low, mid, premium.

```
buckets = [
    any(c == 0 for c in costs),    # free   (₹0)
    any(0 < c <= 500 for c in costs), # low    (₹1-500)
    any(500 < c <= 2000 for c in costs), # mid    (₹501-2000)
    any(c > 2000 for c in costs),    # premium (₹2000+)
]

cost_score = sum(buckets) / 4    # how many buckets are present
```

Example:

- Costs: [0, 300, 800, 1500] → all 4 buckets filled → score = $4/4 = 1.0$
- Costs: [0, 0, 0, 0] → only free bucket → score = $1/4 = 0.25$

No LLM. Just range comparisons on numbers.

Check 3 — Language Bias (30%)

What it does: Scans the entire itinerary text for gendered or exclusionary phrases. Starts at 1.0 and deducts for each match.

```
gendered      = ["ladies only", "men only", "for women", "husband",  
"wife only", ...]  
exclusionary   = ["avoid locals", "unsafe area", "only for rich", ...]
```

```
text = str(itinerary).lower()  # converts entire itinerary dict to  
one string
```

```
lang_score = 1.0  
for term in gendered:  
    if term in text:  
        lang_score -= 0.15      # -15% per gendered term found  
for term in exclusionary:  
    if term in text:  
        lang_score -= 0.20      # -20% per exclusionary term found
```

Example:

- Clean plan → score = 1.0
- Plan says "ladies only spa" → score = $1.0 - 0.15 = 0.85$
- Plan says "avoid locals, men only" → score = $1.0 - 0.20 - 0.15 = 0.65$

No LLM. Just `if term in text` substring matching.

Check 4 — Activity Uniqueness (20%)

What it does: Checks if the LLM lazily repeated the same activity description across days.

```
descriptions = [  
    act.get("description", "").lower()[:40]  # first 40 chars of each  
activity  
    for act in all_activities
```

```
]
unique_ratio = len(set(descriptions)) / len(descriptions)
# set() removes duplicates – if all unique, ratio = 1.0
```

Example:

- 9 activities, all different → 9 unique / 9 total = 1.0
- 9 activities, 3 say "Visit beach and relax" → 7 unique / 9 total = 0.78

No LLM. Just `set()` to count unique strings.

Final Score

```
bias = (0.25 × time_score) + (0.25 × cost_score) + (0.30 × lang_score)
+ (0.20 × unique_ratio)
```

All pure math — **zero LLM calls, zero API cost, runs instantly.**

When would you actually need an LLM for bias?

If you wanted to catch *subtle* bias like:

- *"This destination is better suited for younger travellers"* — implies age bias
- *"Vegetarians may struggle here"* — implies dietary exclusion
- Implicit cultural assumptions in activity suggestions

That kind of nuanced language understanding requires an LLM. What's implemented here catches only **explicit** keyword-level bias, which covers the most obvious cases without any API cost.